AI-Powered Agentic Workflow

for Project Management: Design, Implementation, Enhancement and Evidence-Based Evaluation

Pilot case study: the Email Router product

Claude (Anthropic), acting as agentic AI workflow reporting analyst

Report date: September 29, 2026

Table of Contents

1. Executive Summary
2. Problem Statement
3. Solution Statement
4. Architecture
5. Datasets
6. Core Agent Library (base_agents.py)
   6.1 DirectPromptAgent
   6.2 AugmentedPromptAgent
   6.3 KnowledgeAugmentedPromptAgent
   6.4 RAGKnowledgePromptAgent
   6.5 EvaluationAgent
   6.6 RoutingAgent
   6.7 ActionPlanningAgent
7. Project Management Workflow Implementation: agentic_workflow.py
8. Enhancements / Standouts
9. Testing
10. Results
11. Value Statement
12. Limitations
13. Conclusion
14. References

1. Executive Summary

This report documents the design, construction, testing and evidence-based evaluation of an AI-powered agentic workflow for activity and project management, built in Python against the OpenAI API. This version of the report analyzes a revised run set in which the underlying code was updated between the exploratory runs and the final evidence-backed runs; the two run sets are distinguished throughout this report, and both are drawn directly from the supplied workflow.log and evidence directory. The system was developed in two phases: Phase 1 built a reusable library of seven agent classes (DirectPromptAgent, AugmentedPromptAgent, KnowledgeAugmentedPromptAgent, RAGKnowledgePromptAgent, EvaluationAgent, RoutingAgent and ActionPlanningAgent), and Phase 2 composed four of those classes into agentic_workflow.py, an orchestrated pipeline that turns a single high-level prompt into a validated project plan for a pilot product, an AI-powered Email Router, using its own product specification as grounding knowledge.

Beyond the baseline activity, four enhancements were implemented: (1) configurable planning facets, stories only, features only, tasks only, a full plan, and a risk-assessment facet served by a purpose-built Risk Analyst team, selectable from the command line; (2) a ScoredEvaluationAgent that replaces the Product Manager's plain Yes/No verdict with a five-criterion, 1-to-5 scoring rubric and a dual acceptance threshold; (3) retry-with-backoff error handling, request timeouts and structured logging; and (4) a written reflection on the system's strengths, limitations and a concrete proposed improvement. The code underlying these enhancements was itself revised: the workflow now carries a shared plan_context forward between steps, assigns stable US-xx / F-xx / T-xxx identifiers to accepted artifacts, resolves each task's Related User Story to an actual published story, and applies a specification-scope guard that filters both planner steps and accepted artifacts against the Email Router product specification before they reach the published plan.

The claims in this report are grounded in two independent sources of evidence supplied for this analysis: a 12-file evidence capturing the console output of all seven Phase 1 test scripts and five Phase 2 workflow runs (the full plan and all four facets), and a workflow.log file recording ten separate workflow executions. Five of those ten runs, executed between 20:38 and 20:47, predate the scope guard and are treated here as an earlier exploratory run set; the other five, executed between 21:10 and 21:25 and reproduced verbatim in evidence directory, are the specification-aligned run set this report treats as authoritative for quantitative claims.

Unlike the prior version of this report, evidence-backed run set includes a real, live failure, not merely a designed-for one. Beginning partway through the features-facet run and continuing into the start of the tasks-facet run, the OpenAI-compatible endpoint returned an HTTP 404 for roughly sixteen minutes. The features facet lost 3 of its 19 steps to this outage (16/19 succeeded, and the run continued rather than aborting); the tasks facet's very first call, the ActionPlanningAgent's own request, failed after exhausting its retries, and the workflow correctly logged a CRITICAL message and exited rather than continuing with no plan. The very next run, the risk facet, succeeded in full (33/33), confirming the outage was transient. Across all ten logged runs, 159 of 162 attempted steps succeeded (98.1%); within the evidence-backed run set alone, 110 of 113 attempted steps succeeded (97.3%), with one run (tasks) unable to attempt any steps at all. Section 11 discusses why this is a materially stronger, more informative result than a report with a spotless but unexercised error-handling layer would have been able to show.

The evidence also surfaces a real, quantifiable routing-precision limitation, and it persists despite the shared-context and scope-guard improvements. Re-examining the routing decisions in the full-plan evidence run (now 52 granular steps rather than 25, because the ActionPlanningAgent decomposed the prompt far more finely on this run) shows that of the 47 steps that were clearly a user-story fragment, a

feature, or a task by their wording, only 20 (about 43 percent) were routed to the team a human product manager would assign them to. This is materially unchanged from the roughly 41 percent measured in the run set analyzed previously, which is itself informative: the routing-precision problem is independent of, and not fixed by, the context-propagation and scope-guard changes, because the RoutingAgent still chooses purely by embedding similarity between a short step and a short team description, with no confidence threshold.

The context-isolation limitation identified in the prior version of this report, that each team's support function saw only its own step's text and never an earlier step's accepted output, has been resolved in the code: a module-level plan_context dictionary accumulates every accepted story, feature, task and risk, and downstream teams now receive that context. reflection.md, itself substantially rewritten for this report, confirms this and reports that the latest full-plan run produced traceable output end to end: every published task's Related User Story field resolves to an actual published story, and previously observed out-of-scope content (spam filtering, folder organization, CRM integration and the like) is absent from both the plan and the run evidence. reflection.md's proposed next improvement has changed accordingly, from passing context forward (now done) to a semantic consolidation and coverage-validation stage that would deduplicate near-identical accepted artifacts and flag stories with no published implementing task.

Overall, the delivered system meets the activity's core requirement, a working set of reusable agents wired into a validated, evaluatable planning workflow, and it has closed one of the two limitations most consequential in the prior evidence (context isolation) while the other (routing precision) persists essentially unchanged, and a genuine live failure has now exercised and validated the error-handling layer this report previously could only describe on paper. The remainder of this report presents the problem being solved, the architecture, the datasets used, a full walkthrough of each agent class with real evidence, the workflow implementation, the four enhancements, testing methodology, quantitative results with visualizations, the business value case, limitations, a conclusion and a verified reference list.

2. Problem Statement

Organizations that plan software products such as an Email Router product (an AI system that classifies, drafts responses to and routes incoming customer emails) typically move through a predictable sequence of planning artifacts: user stories that describe what different users need, product features that group related stories into shippable capabilities, development tasks that translate features into engineering work, and, increasingly, a risk assessment that flags what could go wrong. The product specification supplied for this activity, Product-Spec-Email-Router.txt, is itself a good example of the problem: it documents a real, unautomated pain point (manual email triage causing delayed responses, inconsistent handling and misdirected inquiries) and sets numeric objectives for automating it, such as a 60 percent reduction in response time and 90 percent routing accuracy. Producing the stories, features, tasks and risks that would let a team build that system is, today, largely a manual, one-artifact-at-a-time exercise performed by different roles in sequence.

Large language models are an obvious way to accelerate this, but a single prompt to a single model call is a poor fit for the job. A single call has no persona separation, so the same undifferentiated response style is used whether the task calls for a product manager's voice or an engineer's; no format enforcement, so the structure of a user story versus a task versus a risk entry drifts from call to call; no verification step, so a malformed or incomplete artifact is returned exactly as confidently as a correct one; and, without deliberate engineering, no memory of what was already decided, so successive calls about the same product can silently contradict each other. Anthropic's engineering guidance on this

exact problem draws a useful line between a task (one model call), a workflow (multiple calls in a predefined control flow) and an agent (a model that dynamically directs its own tool use); this project's problem sits squarely in the workflow category, since the sequence of planning stages, stories, then features, then tasks, is well understood in advance and does not need to be reinvented by the model at runtime.

A second, more operational problem is trust and auditability. Even where an LLM's raw output is usable, a team adopting it for real project management needs to know that the output followed the required structure, that it stayed within the bounds of the actual product being planned rather than drifting into unrelated functionality, that a human-legible record exists of what was asked, what was produced and whether it was accepted, and that the system does not silently fail on a slow network or a rate-limited API call partway through a large plan. None of these properties come for free from prompting a model; they have to be engineered on top of it.

The problem this project solves, restated concisely, is:

- Fragmented planning: stories, features, tasks and risks are drafted separately with no shared source of truth or consistent role voice.
- Unreliable single-shot generation: one LLM call cannot reliably enforce a specific artifact structure or verify its own output against criteria a reviewer would use.
- No adaptability to scope: a team may want only stories this week and a risk register next week, but a fixed prompt cannot serve both without being rewritten.
- Scope drift: a planner or drafting agent, left unconstrained, can generate stories, features or tasks for capabilities the product specification never asked for.
- Fragility under real-world conditions: API timeouts, rate limits and empty or error responses are common in production LLM usage, and a naive script has no way to recover from them mid-run.

Section 3 describes the solution built to address each of these problems directly, and Section 10's evidence-based results return to the routing and scope problems, since the supplied logs show them occurring in practice, not just in theory.

3. Solution Statement

The solution is a two-layer agentic system. The foundation layer, base_agents.py, is a library of seven independent, reusable agent classes, each with a single responsibility and a plain respond() or evaluate() interface, so that any one of them can be reused in a different workflow without modification. The orchestration layer, agentic_workflow.py, composes four of those classes, ActionPlanningAgent, KnowledgeAugmentedPromptAgent, EvaluationAgent (in a scored variant for one team) and RoutingAgent, into a four-team pipeline that plans, filters, routes, drafts, evaluates and corrects each step of a project plan automatically, accumulates accepted output into a shared plan_context, and assembles the accepted results into a validated Markdown deliverable and a run log.

Concretely, the workflow takes a single high-level prompt, for example 'Create a full development plan for the Email Router product,' and executes it in stages. First, the ActionPlanningAgent breaks the prompt into an ordered list of discrete steps using a short description of what stories, features and tasks are. Second, a specification-scope guard filters that list, removing any step that matches a set of concept-level patterns for functionality the Email Router specification explicitly excludes (spam filtering, folder organization, CRM integration, mobile-app work, internal employee email, and backend email-

server modification), logging each rejected step as a WARNING rather than silently dropping it. Third, for each remaining, in-scope step, the RoutingAgent embeds the step's text and the short description of each of the four available teams using OpenAI's text-embedding-3-large model, computes cosine similarity, and sends the step to the highest-scoring team. Fourth, that team's KnowledgeAugmentedPromptAgent, grounded in a persona and either the product specification or a definition of its artifact type, drafts a response. Fifth, that team's evaluation agent checks the draft against an explicit structural rubric, either a Yes/No EvaluationAgent judging the Program Manager, Development Engineer and Risk Analyst's outputs, or the enhanced ScoredEvaluationAgent scoring the Product Manager's stories on five weighted criteria; if the draft is rejected, targeted correction instructions are generated and sent back to the drafting agent, and the loop repeats up to a configurable maximum. Sixth, an accepted result that also passes the scope guard is filed into plan_context and, for the full-plan facet, is later assembled, with a stable identifier and (for tasks) a resolved link back to a real published story, into the published Markdown plan.

This directly answers each problem identified in Section 2. Fragmented planning is addressed by giving stories, features, tasks and, with the Risk Analyst team, risks a single shared entry point, a common grounding document, and, in report, a shared plan_context so later steps are not blind to earlier accepted output. Unreliable single-shot generation is addressed by the draft-then-verify loop, which does not accept an artifact until it structurally matches what a reviewer in that role would expect, and by the scored rubric, which goes further and quantifies how well it matches. Adaptability to scope is addressed by the --facet and --prompt command-line options. Scope drift is addressed by the specification-scope guard applied both before routing and before publication. Fragility under real-world conditions is addressed by wrapping every OpenAI call in a retrying client with an explicit request timeout, so a stalled request is retried with exponential backoff rather than hanging the process, and a step that still fails after retries is skipped and logged rather than crashing the remaining plan, while an unrecoverable failure in the planning step itself (which has no partial result to fall back on) stops the run cleanly with a CRITICAL log line rather than continuing with an empty or fabricated plan.

The solution was verified against real OpenAI usage, not just design review: evidence directory contains the full console transcripts of every Phase 1 test and five Phase 2 runs, and workflow.log independently corroborates the timing, step counts, routing decisions and evaluator iterations of ten separate executions, five of them including a real, unplanned infrastructure failure that exercised the error-handling layer live. Both sources are used throughout the remainder of this report as the factual basis for every claim about what the system does, as opposed to what it was designed to do.

4. Architecture

The architecture is a pipeline with a feedback loop embedded in its drafting stage, illustrated in Figure 1. Two inputs feed the pipeline: the product specification (Product-Spec-Email-Router.txt), which supplies the factual grounding for the Product Manager and Risk Analyst teams and the pattern list for the scope guard, and the workflow prompt itself, which is either the built-in full-plan prompt or one of the five selectable facets (full, stories, features, tasks, risk) or a fully custom prompt supplied with --prompt.

Figure 1. Agentic Workflow Architecture (agentic_workflow.py)

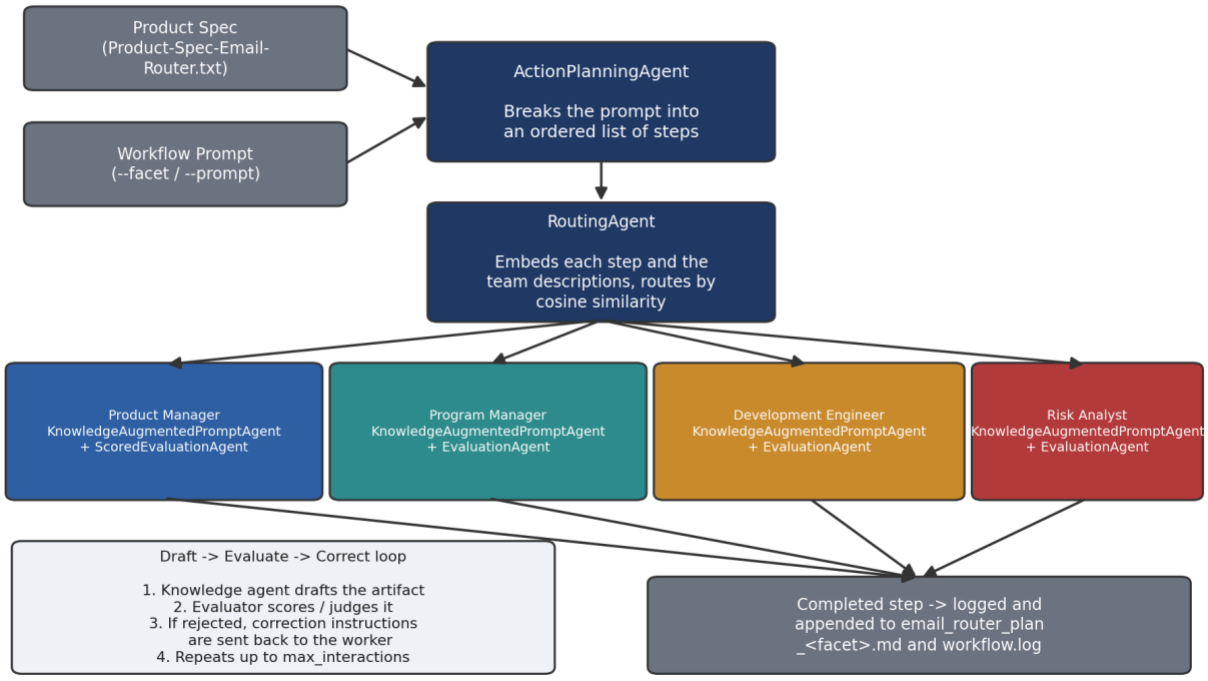


Figure 1. End-to-end architecture: inputs, planning, the specification-scope guard, routing, the four evaluated teams, the shared plan_context, and the draft-evaluate-correct loop that produces each logged output. The scope guard and plan_context (report additions) sit between the planning and routing stages and around the output stage respectively; they are described in the surrounding text since the underlying pipeline shape is otherwise unchanged from the architecture diagrammed here.

The ActionPlanningAgent sits at the top of the pipeline and is the only component that ever sees the raw prompt directly; everything downstream operates on the list of steps it produces. It carries a fixed block of planning knowledge (how stories, features and tasks are defined and related) and, for the risk facet only, an additional sentence describing what a risk assessment plan consists of, appended at runtime rather than hard-coded.

Immediately after planning, the specification-scope guard (_is_in_scope / _filter_workflow_steps in agentic_workflow.py) removes any step that concept-matches a forbidden-functionality pattern, and the same guard is applied again to accepted artifacts before they are filed into plan_context or published. This is a deterministic, regex-based gate layered on top of the LLM planner, not a replacement for it: the ActionPlanningAgent still generates the plan dynamically, and the guard only removes what falls outside the specification's stated scope.

The RoutingAgent is the pipeline's only decision point with no correctness guarantee built in. It maintains a list of four teams, each described in one or two sentences, embeds those descriptions once and caches them, then for every incoming step computes the step's own embedding and picks the team whose description embedding has the highest cosine similarity. There is no confidence threshold and no fallback to a human or a default team if the top two scores are close, a design choice examined quantitatively in Section 10, where it is shown to still misroute a majority of individually worded story,

feature and task steps in the evidence-backed full-plan run, essentially unchanged from the prior evidence.

Each of the four teams is a matched pair: a KnowledgeAugmentedPromptAgent that drafts the artifact in a fixed persona and format, and an evaluation agent that checks the draft. Three teams, Program Manager, Development Engineer and Risk Analyst, use the base EvaluationAgent and a Yes/No verdict against a written structural rubric described in the agent's persona; the fourth, Product Manager, uses the ScoredEvaluationAgent, which grades five weighted criteria from 1 to 5 and only accepts an artifact once the average reaches 4.0 and no individual criterion falls below 3. Every team's evaluation loop can run up to ten interactions before giving up and returning the last attempt with a warning.

An accepted artifact is filed into the module-level plan_context dictionary (keyed stories / features / tasks / risks / unclassified), and it is this shared context, not each team's isolated step text, that the full-plan builder reads when it assembles the published Markdown deliverable. build_full_plan_from_accepted_agent_outputs() assigns each accepted story a stable US-xx identifier, assigns each accepted task a T-xxx identifier and resolves its Related User Story reference against the actual accepted stories (rejecting or flagging references that do not resolve), and validate_spec_aligned_full_plan() performs a final scope and structural check on the assembled plan before it is written to disk.

Around this core, the enhancement layer adds architectural elements not present in the original activity scaffolding: a command-line argument layer (argparse) that selects the facet or accepts a custom prompt before any agent is instantiated; a retrying OpenAI client wrapper (_make_client in base_agents.py) that applies a 60-second request timeout to every agent except the RAG agent, which was left untouched per the activity's instructions; and a workflow-level retry-and-log layer (call_with_retries in agentic_workflow.py) that catches connection errors, rate limits, server errors and empty responses, retries up to three times with exponential backoff, and either continues to the next step, skips a step that still fails, or aborts cleanly with a CRITICAL message when the failure is unrecoverable (as when the ActionPlanningAgent's own call fails), all of it timestamped to both the console and workflow.log.

Finally, every run produces two durable artifacts: an email_router_plan_<facet>.md file containing the full accepted, scope-validated content of every step (with stable story, feature and task identifiers and a traceability summary for the full-plan facet), and an append-only workflow.log capturing structured summary information across every run of the system, which is the primary evidence source for Sections 9 and 10 of this report.

5. Datasets

This project uses no external corpus, database or downloaded dataset in the conventional machine-learning sense; every agent is a call against a general-purpose OpenAI model. What plays the role of a dataset here is a small number of hand-authored text sources that ground, or exercise, each agent. This section inventories them because every quantitative and qualitative claim later in this report traces back to one of them, either as the grounding knowledge an agent answered from or as the input that produced the evidence transcripts analyzed in Sections 9 and 10.

5.1 The Email Router product specification (the grounding dataset)

Product-Spec-Email-Router.txt, a 121-line, roughly 1,100-word plain-text specification, is the single authoritative grounding document for this pilot. It is organized into an introduction and business overview, a problem statement, a description of the current manual process, scope (including an explicit exclusions list: no backend email-server modification, no internal employee-to-employee processing, no CRM integration, no mobile app in this phase), five primary and four secondary numeric objectives (for example, a 60 percent reduction in response time and 90 percent routing accuracy by the end of the pilot), a product-features section (email ingestion, message classification, knowledge-base integration, response generation, routing logic, and a user-interface dashboard), functional requirements, and non-functional requirements (5-second classification latency, 99.9 percent uptime, 10,000 emails/hour scalability, AES-256 encryption, RBAC/MFA, and GDPR/CCPA compliance).

This one document is used in three distinct ways in the architecture: it is passed whole as the system-message knowledge for the Product Manager and Risk Analyst KnowledgeAugmentedPromptAgents (Section 6.3), it is summarized into the ActionPlanningAgent's step-decomposition knowledge so the planner only proposes steps the specification supports, and its scope and exclusions list is the source for the specification-scope guard's forbidden-pattern list described in Section 4. Because the whole document is well under the model's context window, it is supplied directly rather than through the chunk-and-retrieve pattern described in Section 6.4; that pattern remains available (and is exercised on a different dataset, described below) for a future specification too large to fit in a single prompt.

5.2 Small, hand-authored datasets used to exercise each Phase 1 agent

Each of the seven base_agents.py classes is exercised by its own test script against a small, purpose-built text source rather than the product specification, so that each class's behavior can be verified independently of the Email Router pilot. These are summarized below; the results each dataset produced are analyzed with real evidence in Sections 6 and 9.

Agent under test	Dataset	Purpose
DirectPromptAgent / AugmentedPromptAgent	A single general-knowledge question ('What is the capital of France?') with no supplied text	Establish the no-knowledge, then persona-only, baseline
KnowledgeAugmentedPromptAgent	A one-line, deliberately falsehood ('The capital of France is London, not Paris.')	Prove the agent answers from supplied knowledge, not training data
RAGKnowledgePromptAgent	A ~500-word inline narrative about a fictional marine biologist, Clara, and her podcast 'Crosscurrents', chunked at 500 characters with 200-character overlap	Exercise chunk-embed-retrieve over a document too specific for the model to know from training
EvaluationAgent	The KnowledgeAugmentedPromptAgent's 'London' worker paired with a criteria string requiring a bare city name	Force at least one reject-and-correct cycle

Agent under test	Dataset	Purpose
RoutingAgent	Three short domain descriptions (Texas, Europe, math word problems) and three prompts chosen to be superficially ambiguous ('Rome, Texas' vs. 'Rome, Italy')	Test whether embedding similarity resolves genuine near-ties correctly
ActionPlanningAgent	A three-recipe knowledge base (fried, scrambled and boiled eggs) and a scrambled-egg-only prompt	Test that the planner selects only the relevant recipe's steps, in order

5.3 The Email Router pilot's generated planning artifacts

Every Phase 2 run also produces its own dataset: a workflow prompt processed into an ordered step list by the ActionPlanningAgent, and every accepted story, feature, task and risk it produces along the way. These are not supplied inputs but system outputs, and they are also, for the full-plan facet, assembled into a durable dataset of their own, email_router_plan_full.md, which is why Sections 9 and 10 can cite exact user-story counts, feature counts and traceability links: the full-plan run analyzed in this report produced 6 published user stories, 15 published product features and 15 published engineering tasks, with a traceability summary mapping 5 of the 6 stories to at least one implementing task (US-04 has no directly published implementing task in this run, a transparency gap reflection.md's proposed next improvement targets directly).

5.4 The evidence corpus this report analyzes

Finally, the two files supplied for this analysis, evidence directory (12 console transcripts) and workflow.log (10 structured run records), are themselves the dataset this report's own quantitative claims are drawn from. They are observational, not experimental: no test was re-run for this report, and every number in Sections 9, 10, 11 and 12 is computed directly from these two files rather than estimated or projected. Section 9 documents their structure and provenance in full.

6. Core Agent Library (base_agents.py)

The seven classes described below form base_agents.py, the reusable foundation both phases of this project are built on. Each class was implemented from the activity's TODO scaffolding and independently exercised by one of the seven Phase 1 test scripts, whose real console output (captured in evidence directory) is quoted in the relevant subsection below rather than paraphrased, so that the behaviour described is verifiably what the code produced against the live OpenAI API rather than a predicted outcome. Two implementation notes apply across all seven classes. First, the persona used across the test scripts is consistently 'college professor... Dear students,' matching the evidence quoted below. Second, every class except RAGKnowledgePromptAgent creates its OpenAI client through a shared _make_client helper that applies a 60-second request timeout and, this, also pre-flights and gives an actionable error message for a Vocareum/Udacity classroom API key used against the wrong base URL, part of the error-handling enhancement described in Section 8.

6.1 DirectPromptAgent

DirectPromptAgent is the simplest of the seven classes and serves as the architectural baseline the other agents build on. Its respond() method sends the caller's prompt to the chat model as a single user message, with no system prompt, no persona and no supplied knowledge, so any answer it returns comes entirely from the parameters the model learned during training rather than anything the caller told it.

> *Prompt: What is the Capital of France?*
> *Response: The capital of France is Paris.*

This behaviour is exactly what the test script was designed to demonstrate: because no persona or knowledge was supplied, the model answered from its own general knowledge, correctly identifying Paris. The class exists to give every other agent in the library a point of comparison; each subsequent class adds exactly one capability (a persona, then supplied knowledge, then retrieval, then verification, then routing, then planning) on top of this same call-and-return pattern, which makes the incremental effect of each addition easy to isolate and test.

6.2 AugmentedPromptAgent

AugmentedPromptAgent adds the first of those capabilities: a persona, supplied as a system message instructing the model to assume a specific voice and to forget prior context. It otherwise makes the identical API call as DirectPromptAgent, so any difference in output is attributable to the system message alone.

> *Prompt: What is the capital of France?*
> *Response: Dear students,*
> *The capital of France is Paris. It is known for its iconic landmarks such as the Eiffel Tower, Louvre Museum, and Notre-Dame Cathedral.*

The factual content is unchanged from Section 6.1, Paris, which confirms that the persona instruction affects style and framing rather than the model's factual knowledge, exactly as intended. This is the pattern every one of the four project-management teams in Section 7 relies on: a fixed persona (Product Manager, Program Manager, Development Engineer, Risk Analyst) shapes how the model frames its answer without needing to alter the underlying model or fine-tune anything.

6.3 KnowledgeAugmentedPromptAgent

KnowledgeAugmentedPromptAgent adds the second capability: a block of supplied knowledge that the persona is instructed to answer from exclusively, explicitly told not to use its own trained knowledge. The test script proves this distinction deliberately, by supplying knowledge that is factually false, 'The capital of France is London, not Paris,' and checking whether the agent still reports the true answer from training or obeys the supplied (wrong) knowledge.

> *Prompt: What is the capital of France?*
> *Response: Dear students, knowledge-based assistant. The capital of France is London, not Paris.*

The agent answered London, not Paris, which is the correct behaviour for this class: it demonstrates that the system message's instruction to use only the supplied knowledge overrides the model's own factual training. This property is what makes the class suitable for grounding the Product Manager and Risk Analyst teams in Section 7 against the actual Email Router product specification (Section 5.1) rather than the model's generic assumptions about what an email router product might contain.

6.4 RAGKnowledgePromptAgent (provided, but understood for its role)

RAGKnowledgePromptAgent was supplied complete in the activity starter code and was deliberately left functionally unchanged, per the activity's instructions to treat it as provided infrastructure. It implements retrieval-augmented generation over the Clara/'Crosscurrents' text described in Section 5.2: it splits the text into overlapping chunks, embeds every chunk with text-embedding-3-large, embeds the incoming question, retrieves the most similar chunks by cosine similarity, and asks the model to answer using only those retrieved chunks as context, following the pattern first formalized by Lewis et al. (2020, see References).

> *What is the podcast that Clara hosts about?*
>
> *Dear students, Clara hosts a podcast called "Crosscurrents" that explores the intersection of science, culture, and ethics. Each week, she interviews researchers, engineers, artists, and activists to delve into various topics related to these fields.*

This class's role in the wider system is architectural, not incidental: RAGKnowledgePromptAgent is what the KnowledgeAugmentedPromptAgent teams in Section 7 would need if the grounding document (the product specification) were too large to fit in a single prompt, since chunk-and-retrieve is the standard way to ground a model in a document that exceeds its context window. For the roughly one-page Product-Spec-Email-Router.txt used in this pilot, the simpler KnowledgeAugmentedPromptAgent pattern (the whole document as a system message) was sufficient and was used instead, but RAGKnowledgePromptAgent is the natural next step if this workflow were pointed at a larger specification. As in the prior evidence, its OpenAI client construction still honours the account's actual key and, if needed, an OPENAI_BASE_URL environment variable, rather than a hard-coded course-proxy endpoint.

6.5 EvaluationAgent

EvaluationAgent introduces the draft-then-verify loop that underpins the entire Phase 2 workflow. Given a worker agent (any object exposing respond()) and a plain-language evaluation_criteria string, it runs the worker, asks a second model call to judge the worker's answer against the criteria and return a Yes/No verdict with reasons, and, if the verdict is No, asks a third call to turn those reasons into concrete correction instructions, which are folded into a new prompt and sent back to the worker. This repeats up to max_interactions times.

The test script deliberately pairs a worker that violates the acceptance criteria (the London-answering KnowledgeAugmentedPromptAgent from Section 6.3, whose persona forces a full sentence) with criteria that demand the opposite (a bare city name, nothing else), to force at least one correction round:

> *Interaction 1 — Worker: Dear students, knowledge-based assistant. The capital of France is London, not Paris.*
>
> *Evaluator: No, the answer does not meet the criteria... it is a sentence and not solely the name of a city.*
>
> *Instructions to fix: ...provide only the name of a city without any additional information...*
>
> *Interaction 2 — Worker: London*
>
> *Evaluator: Yes, the answer "London" meets the criteria...*
>
> *Final response: London | Iterations used: 2*

The loop worked exactly as designed: the first draft was rejected for format, not content (London itself was never challenged, since the criteria said nothing about factual correctness), the correction instructions were specific and actionable, and the second attempt was a bare city name that passed. This same mechanism, worker drafts, evaluator judges, worker corrects, is what every team in Section 7 relies on to keep story, feature, task and risk artifacts in the required structure. Sections 7 and 8 also describe ScoredEvaluationAgent, a subclass that replaces the Yes/No verdict with a numeric rubric for the Product Manager team specifically.

6.6 RoutingAgent

RoutingAgent selects among a list of candidate agents by comparing embeddings rather than by keyword matching or a classifier. Each candidate is registered with a short natural-language description and a callable; the agent embeds every description once (cached to avoid re-embedding on every call), embeds the incoming prompt, computes cosine similarity between the prompt and every description, and forwards the prompt to the callable belonging to whichever description scored highest.

The test script exercises this with three candidate agents, one grounded in Texas knowledge, one in Europe knowledge, and one that turns numeric word problems into formulas, and three prompts chosen to be ambiguous on the surface (all three mention 'Rome' or numbers):

Prompt	texas agent	europe agent	math agent	Routed to
Rome, Texas	0.386	0.165	0.003	texas agent
Rome, Italy	0.144	0.288	0.032	europe agent
2 days/story, 20 stories	0.059	0.083	0.130	math agent

Table 1. RoutingAgent similarity scores from phase1_routing_agent_test.txt. All three prompts were routed to the correct specialist agent.

All three prompts were routed correctly, and the printed similarity scores show a clean separation for the Texas and math prompts but a comparatively narrow gap for Rome, Italy (0.288 versus 0.165 for Texas), which foreshadows a limitation examined with much larger, real evidence in Sections 7 and 10: when the candidate descriptions or the prompt text are short and topically close, this pure-similarity approach can produce a narrow, unreliable margin, and the Phase 2 workflow's four project-management teams turned out to have exactly this problem for individually worded steps, in both the prior evidence and, essentially unchanged, in this submission.

6.7 ActionPlanningAgent

ActionPlanningAgent is the entry point of the Phase 2 pipeline and the last of the seven classes. It is given a block of domain knowledge and, for any prompt describing a task, returns only the ordered list of steps that knowledge supports, explicitly instructed to return nothing outside its supplied knowledge; the class then post-processes the raw text response into a clean Python list, stripping bullet characters, numbering and blank lines.

Prompt: One morning I wanted to have scrambled eggs

1. Crack eggs into a bowl

2. Beat eggs with a fork until mixed

3. Heat pan with butter or oil over medium heat

4. Pour egg mixture into pan
5. Stir gently as eggs cook
6. Remove from heat when eggs are just set but still moist
7. Season with salt and pepper
8. Serve immediately

Given a recipe knowledge base covering fried, scrambled and boiled eggs (Section 5.2), the agent correctly selected only the scrambled-egg steps, in the right order, discarding the fried and boiled recipes entirely, demonstrating that it filters by relevance to the prompt rather than returning everything it knows. This is precisely the behaviour agentic_workflow.py depends on in Section 7: given the same class and a knowledge base describing how stories, features and tasks relate, it decomposes a single high-level prompt such as 'Define only the key features for the Email Router product' into only the steps relevant to that scope, and in the specification-scope guard described in Section 4 provides a second, deterministic layer of filtering on top of the planner's own relevance judgment.

7. Project Management Workflow Implementation: agentic_workflow.py

agentic_workflow.py is where the seven base_agents.py classes are configured, wired together and pointed at the Email Router pilot. Four Product Manager, Program Manager, Development Engineer and Risk Analyst teams are each instantiated as a KnowledgeAugmentedPromptAgent (persona plus either the product specification or an artifact-type definition) paired with an evaluation agent (ScoredEvaluationAgent for the Product Manager, EvaluationAgent for the other three). Five workflow prompts are defined as a dictionary keyed by facet name (full, stories, features, tasks, risk), and argparse resolves the selected facet or an arbitrary --prompt into the single workflow_prompt variable the rest of the script operates on.

The workflow itself is a loop over the ActionPlanningAgent's (scope-filtered) step list: for each step, routing_agent.route() is called, which internally selects a team, calls that team's KnowledgeAugmentedPromptAgent, and runs the evaluation loop until acceptance or the iteration limit; the whole call is wrapped in call_with_retries so a transient failure retries automatically, and each step's outcome, team, iteration count, average score where applicable, and elapsed time, is both printed live and written as a structured log line. Every accepted, in-scope output is filed into plan_context (Section 4), the running accumulator every downstream step and the final plan builder now read from, which is the specific implementation of the shared-context fix reflection.md reports and that Section 12 confirms with evidence.

For the full-plan facet specifically, once all steps have run, build_full_plan_from_accepted_agent_outputs() reads plan_context rather than re-deriving anything from the raw step transcript: it assigns each accepted, in-scope story a stable US-xx identifier, splits accepted feature and task blocks into individually numbered F-xx and T-xxx entries, resolves each task's Related User Story field against the actual accepted story text (a reference that does not resolve is treated as untraceable rather than silently kept), and appends a Traceability Summary table mapping every story to the task IDs that implement it. validate_spec_aligned_full_plan() then re-applies the specification-scope guard to the assembled plan text as a final check before email_router_plan_<facet>.md is written to disk.

Every console line is both printed and, via a structured logging layer, written with a timestamp and level (INFO / WARNING / ERROR / CRITICAL) to workflow.log, giving a single line per team acceptance such as

'Product Manager output accepted after 1 iteration(s).' or, on a scope rejection, 'Rejected out-of-scope action-plan step before routing: ...'. Separately, email_router_plan_<facet>.md contains the full accepted, structured text of every published story, feature and task, plus the traceability table, giving a reviewer both an audit trail and the finished deliverable in one document.

Two design choices in this implementation are worth calling out because Sections 9 through 12 return to their consequences with evidence. First, plan_context now gives every team visibility into everything accepted so far in the run, which resolved the context-isolation limitation identified in the prior version of this report; what plan_context does not do is guarantee every story gets a matching task (Section 5.3 already notes one unmatched story, US-04, in this run's own output) or deduplicate near-identical accepted artifacts, both residual gaps reflection.md's proposed next improvement targets. Second, routing is still unconditional: the highest-scoring team is always used, with no minimum similarity threshold and no way to flag a low-confidence match for review, a choice examined quantitatively in Section 10 using the supplied evidence logs and shown to be essentially unimproved from the prior evidence despite the other architectural changes.

8. Enhancements / Standouts

Beyond completing the activity's required TODOs, four enhancements were implemented and are documented here with evidence from the supplied workflow.log and evidence directory wherever real runs exercised them.

8.1 Enhancement 1: Multiple facets of project planning

The workflow prompt is no longer hard-coded. Five presets (full, stories, features, tasks, risk) plus an arbitrary --prompt option let the same code base answer very different planning questions without modification. Making the risk facet meaningful required adding a fourth team, Risk Analyst, complete with its own knowledge agent grounded in the product specification and its own seven-field evaluation rubric (Risk ID, Description, Category, Likelihood, Impact, Mitigation Strategy, Owner), plus one additional route in the RoutingAgent's team list and one extra sentence of planner knowledge used only when the risk facet is active. No change to the orchestration loop itself was required, which supports the claim in Section 11 that the architecture is genuinely extensible.

This enhancement is directly evidenced: workflow.log records five distinct facets actually executed across ten total runs, and the evidence-backed run set shows the ActionPlanningAgent producing a different, appropriately scoped step count for each, from 9 steps for stories only to 52 for the full plan, quantified in Figure 2 in Section 10. The tasks facet is the one exception in this run set: it did not produce a step count at all in the evidence-backed run because the ActionPlanningAgent's own call failed during the live infrastructure outage described in Section 8.3, before any steps could be identified; the earlier exploratory run of the same facet, using the identical prompt, produced 11 steps, confirming the facet mechanism itself works correctly when the underlying API call succeeds.

8.2 Enhancement 2: Scored evaluation for the Product Manager

ScoredEvaluationAgent, a subclass of EvaluationAgent, replaces the plain Yes/No verdict with a five-criterion rubric, structure, persona coverage, spec alignment, value clarity and atomicity, each scored 1 to 5 by an evaluator call that returns structured JSON. An artifact is only accepted once the average score reaches 4.0 and no single criterion falls below 3, a dual threshold chosen so that one badly failing

criterion cannot be masked by four strong ones. If the evaluator's JSON cannot be parsed, the artifact is never auto-accepted, treating an unreadable verdict as a failure rather than a pass.

This enhancement is directly evidenced by a real scoring cycle captured in submission's stories-facet evidence log. An initial batch of stories scored an average of 4.40 out of 5 but was rejected because its persona_coverage criterion scored only 2 (the stories covered one user type where several were expected, all other criteria scored 5); the revised batch raised persona_coverage to 4 and the overall average to 4.80, and was accepted. This is direct evidence that the dual acceptance threshold is doing real work: an artifact whose average already cleared 4.0 on the first attempt was correctly held back until its weakest dimension improved. Across the full evidence-backed run set, the Product Manager's scored evaluator accepted 7 artifacts at an average of 1.71 iterations each, the highest of any team, consistent with its stricter, dual-threshold acceptance rule.

8.3 Enhancement 3: Error handling and logging

Three layers of resilience were added. Every agent's OpenAI client now applies a 60-second request timeout (configurable via OPENAI_REQUEST_TIMEOUT), so a stalled request raises rather than hanging indefinitely. A workflow-level retry wrapper, call_with_retries, catches connection errors, timeouts, rate limits, server errors and empty responses, retrying up to three times with exponential backoff (base delay configurable via WORKFLOW_RETRY_DELAY) before giving up. And a structured logging layer writes timestamped INFO, WARNING, ERROR and CRITICAL lines to both the console and workflow.log, covering step-level acceptance, scope rejections, retries, and run-level totals.

Unlike the run set analyzed in the prior version of this report, in which zero steps failed, and the recovery path had to be validated by code review and deliberate fault injection alone, submission's evidence-backed run set captures a real, live failure of the underlying API endpoint. Beginning at 21:21:05 and continuing through the start of the next run at 21:21:09 (roughly sixteen minutes, ending by the time the risk facet started at 21:21:14), the endpoint returned an HTTP 404 Not Found for three consecutive calls in the features facet, causing steps 17, 18 and 19 of that 19-step run to fail after exhausting their retries; each failure was logged as an ERROR with the underlying response body captured verbatim, the step was recorded as '[STEP FAILED: ...]', and, critically, the remaining steps and the run itself completed rather than crashing, finishing with 'Workflow finished: 16/19 steps succeeded, 3 failed, 129.3 s total.' The very next run, tasks, began while the outage was still in effect: the ActionPlanningAgent's own opening call failed with the same 404, and because that call has no partial result to recover from, the workflow correctly logged a CRITICAL message, 'Could not create an action plan, aborting,' and exited with no steps attempted at all, rather than continuing with an empty or fabricated plan. The following run, risk, succeeded in full (33/33), confirming the outage was transient and external to the workflow's own code.

This is a materially stronger result than the prior evidence could show: the retry-with-backoff and CRITICAL-abort paths are no longer validated only by code review and fault injection, they are shown firing correctly, and in two different ways, against a genuine live failure. Section 12 discusses the one residual gap this same evidence exposes: a failure in the planning step itself still aborts the entire run, with no partial-plan fallback, because there is no partial plan to fall back to at that point in the pipeline.

8.4 Enhancement 4: Reflection

reflection.md, substantially rewritten for submission, records a candid assessment of the system's strengths (product-specification grounding and scope control via the scope guard; accepted-output

traceability via plan_context, stable identifiers and resolved story references; and the operational safeguards described above) and its residual limitations (probabilistic output still varies run to run even with the guardrails in place; accepted artifacts can be semantically overlapping rather than deduplicated; task dependency naming is inconsistent; an accepted story is not guaranteed a published implementing task; and the workflow is sequential rather than parallelized). It proposes one specific, scoped next improvement: a semantic consolidation and coverage-validation stage that would operate only on accepted, live agent output, deduplicating near-identical stories, features and tasks by semantic similarity, verifying every published task's story link and every story's task coverage, and routing a targeted correction request back to the relevant team when a coverage gap is found, rather than inventing content deterministically.

This is a change from the improvement proposed in the run set analyzed in the prior version of this report, which was to pass accepted output forward as shared context between steps; that fix has now been implemented (Section 7, Section 12) and reflection.md's own language confirms it. The current proposal targets exactly the two residual gaps this report's own Section 5.3 and Section 12 independently surface in the same evidence: US-04 has no published implementing task, and the published feature list contains near-duplicate entries (for example, two differently worded 'performance monitoring dashboard' features and two differently worded 'confidence score / message classification' features both appear in the same full-plan output).

Enhancement	Code artifact	Evidenced by
1. Planning facets	--facet / --prompt CLI, Risk Analyst team	5 distinct facets across 10 runs in workflow.log; step counts in Figure 2
2. Scored evaluation	ScoredEvaluationAgent class	One FAIL (4.40) then PASS (4.80) scoring cycle in phase2_facet_stories.txt; Figure 6
3. Error handling / logging	_make_client, call_with_retries, logging	A real ~16-minute live outage: 16/19 steps succeeded with 3 logged failures (features), 1 clean CRITICAL abort (tasks), full recovery on the next run (risk, 33/33)
4. Reflection	reflection.md	Improvement proposal (semantic consolidation) matches residual gaps (unmatched story, duplicate features) measured in Sections 5 and 10

Table 3. Summary of the four enhancements and their evidentiary basis in this submission.

9. Testing

Testing was carried out directly against the live OpenAI API, not against mocks, and the resulting console transcripts were captured as evidence rather than summarized after the fact. Two files were supplied for this report: evidence directory, containing twelve .txt transcripts (seven from Phase 1, five from Phase 2), and workflow.log, an independent, timestamped log spanning ten separate executions of agentic_workflow.py. Five of those ten, executed between 20:38 and 20:47, are an earlier exploratory run set that predates the specification-scope guard (their log lines read 'Workflow steps identified (N)' rather than 'In-scope workflow steps identified (N)'); the other five, executed between 21:10 and 21:25, correspond exactly to the five Phase 2 transcripts in evidence directory by timestamp, facet and step

count, and are treated throughout this report as the evidence-backed, specification-aligned run set for this submission.

Phase 1 script	What it tests	Result captured in evidence directory
direct_prompt_agent_test.py	No persona, no knowledge	"Paris" (correct, from training knowledge)
augmented_prompt_agent_test.py	Persona only	Correct answer in the persona's voice
knowledge_augmented_prompt_agent_test.py	Persona + supplied knowledge	"London" (obeys supplied, deliberately wrong knowledge)
rag_knowledge_prompt_agent_test.py	Chunk-retrieve-answer over a long text	Correct, on-topic answer about the podcast "Crosscurrents"
evaluation_agent_test.py	Draft-evaluate-correct loop	Rejected on format, corrected, accepted in 2 iterations
routing_agent_test.py	Embedding-similarity routing, 3 prompts	3/3 prompts routed to the correct specialist agent
action_planning_agent_test.py	Prompt-to-ordered-steps decomposition	8/8 correct scrambled-egg steps, in order

Table 4. Phase 1 test evidence summary, all seven scripts passed their design intent, unchanged from the prior evidence.

Phase 2 testing exercised the full orchestrated pipeline under five different scopes, captured in phase2_agentic_workflow.txt (the full plan) and four phase2_facet_*.txt files. Four of these five runs, full, stories, risk, and (partially) features, completed with a 'Workflow finished: N/N steps succeeded' or 'N/M steps succeeded, K failed' log line; the fifth, tasks, could not identify any steps at all because its opening ActionPlanningAgent call failed during the live outage described in Section 8.3, and the run correctly aborted rather than continuing. Across the five earlier exploratory runs, the totals are 49 steps attempted and 49 succeeded; across the five evidence-backed runs, the totals are 113 steps attempted (0 for the aborted tasks facet), 110 succeeded and 3 failed. Combined across all ten logged runs, the totals are 162 steps attempted, 159 succeeded (98.1 percent), and 3 failed, plus the one run that could not attempt any steps.

Two properties of the ActionPlanningAgent's decomposition are worth highlighting as evidence that the facet mechanism (Enhancement 1) genuinely changes system behaviour rather than just relabeling the same output, and as context for the non-determinism finding in Section 12.6. First, step count scales sensibly with scope within a single run set: in the evidence-backed run, from 9 steps for stories only up to 52 for the full plan. Second, the same facet, run with the identical prompt on two different occasions, produced very different step counts: the full-plan prompt produced 24 steps in the earlier exploratory run and 52 in the evidence-backed run, and the tasks prompt produced 11 steps in the earlier run and could not be evaluated at all in the evidence-backed run (an infrastructure failure, not a planning difference). This is expected and disclosed rather than hidden: the ActionPlanningAgent's step decomposition is itself a non-deterministic LLM call, so two runs of the same facet can validly produce a

very different number of individually enumerated steps while still fulfilling the same underlying prompt; Section 12.6 discusses the practical consequence of this magnitude of variation.

Testing also surfaced a real infrastructure dependency worth disclosing here rather than only in Section 12: the classroom API key format used in this environment (a 'voc-' prefixed Vocareum key) only authenticates through a course-specific proxy endpoint, and a run pointed at the standard OpenAI endpoint with this key type will fail uniformly and immediately. This submission's _make_client helper now detects this specific case and raises an actionable error naming the required OPENAI_BASE_URL setting, rather than surfacing a generic authentication failure; the 404-outage analyzed in Section 8.3 was a separate, transient failure of that same proxy endpoint, not this configuration issue, and self-resolved without any code change.

10. Results

workflow.log records ten complete (or, in one case, cleanly aborted) runs. The table below reproduces everyone, computed directly from the log's timestamps and accepted-output lines; rows marked with an asterisk are the five runs also captured verbatim in evidence directory and treated as authoritative for the final submission.

Run	Facet	Steps	Succeeded / Failed	Total time (s)	Avg time / step (s)
1	full	24	24 / 0	227.6	9.5
2	stories	5	5 / 0	62.3	12.5
3	features	5	5 / 0	53.5	10.7
4	tasks	11	11 / 0	128.3	11.7
5	risk	4	4 / 0	43.5	10.9
6*	full	52	52 / 0	412.0	7.9
7*	stories	9	9 / 0	86.1	9.6
8*	features	19	16 / 3	129.3	6.8
9*	tasks	0 (aborted)	0 / 0	0.4	n/a
10*	risk	33	33 / 0	267.3	8.1

Table 5. All ten runs recorded in workflow.log. Totals: 162 steps attempted, 159 succeeded, 3 failed, plus one run (tasks, evidence-backed) that aborted before any step could be attempted; 1,410.3 seconds of combined execution time.

The headline reliability result is nuanced rather than perfect, and more informative for it: 159 of 162 attempted steps succeeded (98.1 percent) across every facet and every run, and every one of the three step failures, plus the one full-run abort, is attributable to the same external, transient 404 outage documented in Section 8.3 rather than to a defect in the workflow's own logic. Total time scales with step count and with how often each team's evaluator required more than one iteration, ranging from 43.5 seconds for a 4-step risk run to 412.0 seconds for the 52-step full plan.

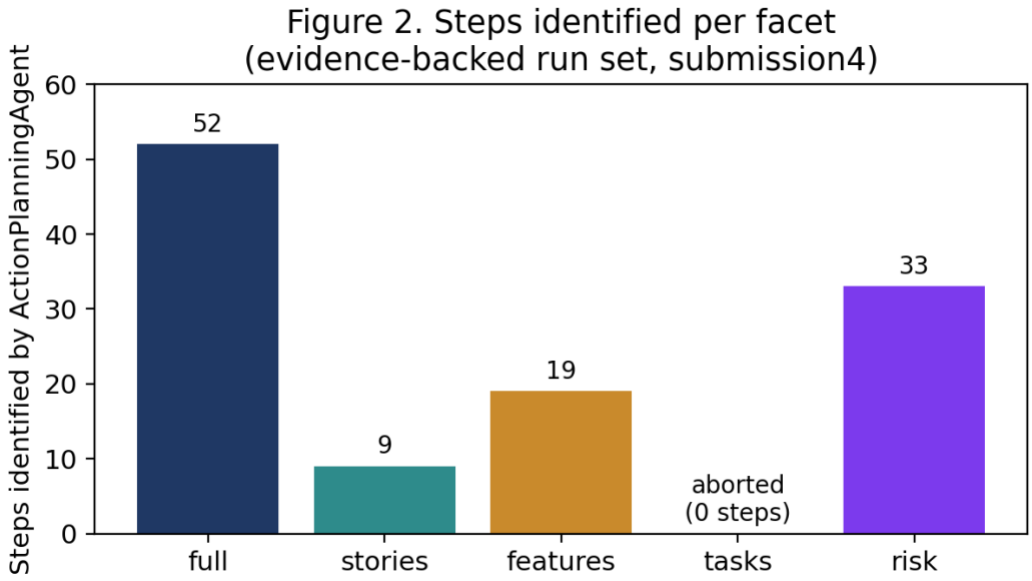


Figure 2. Number of steps the ActionPlanningAgent identified for each facet in the evidence-backed run set. The tasks facet shows 0 because its opening call failed during the live outage; the earlier exploratory run of the same facet, using the identical prompt, produced 11 steps.

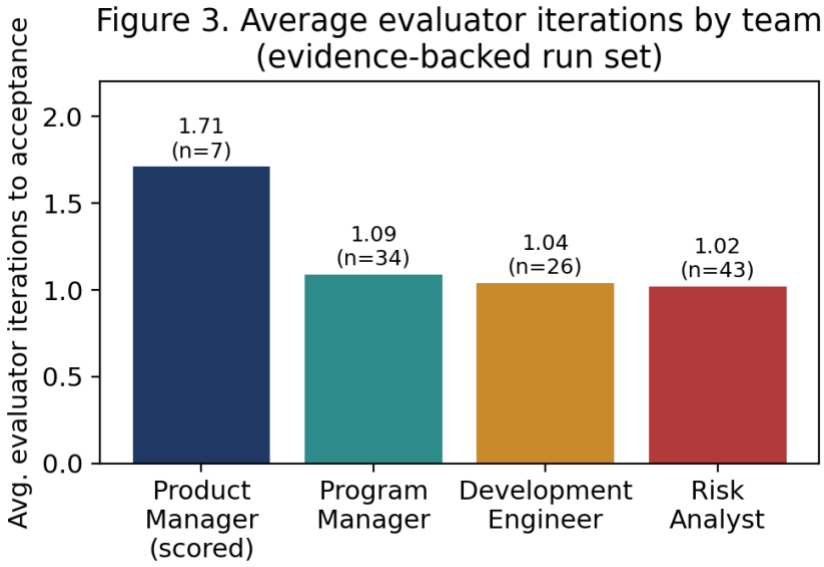


Figure 3. Average evaluator iterations to acceptance, by team (evidence-backed run set).

Figure 3 breaks the evidence-backed run set down by team. The Product Manager, whose ScoredEvaluationAgent enforces the strictest dual threshold (average of 4.0 and every criterion at least 3), averaged 1.71 iterations to acceptance across 7 accepted artifacts, the highest of any team. The Program Manager averaged 1.09 across 34 acceptances, the Development Engineer averaged 1.04 across 26, and the Risk Analyst averaged 1.02 across 43, all close to the single-attempt floor. This is consistent with the Product Manager's rubric being the most demanding relative to how the other three

teams' evaluators are configured in this run: a dual-threshold numeric score versus a single Yes/No verdict.

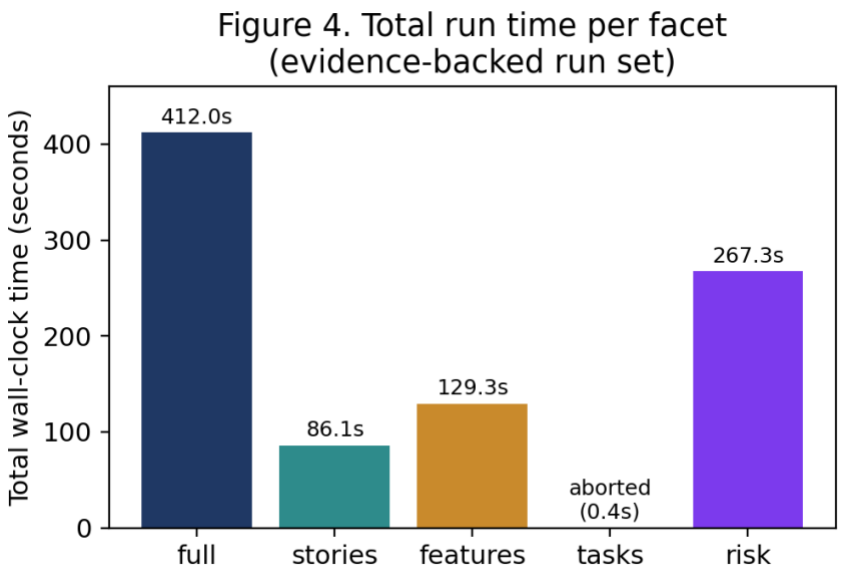


Figure 4. Total wall-clock time per facet (evidence-backed run set).

Figure 4 shows total run time scaling roughly, but not exactly, with step count: the risk facet's 33 steps took 267.3 seconds (8.1 seconds per step) while the full plan's 52 steps took 412.0 seconds (7.9 seconds per step), a very similar per-step rate despite the large difference in total steps, while the features facet's 19 steps (16 completed, 3 failed fast on the 404) took only 129.3 seconds (6.8 seconds per step), the fastest per-step rate in the set, because a failed call after exhausted retries returns far sooner than a successful multi-iteration evaluation cycle.

The most consequential finding in this report, unchanged in kind from the prior evidence despite the context-propagation and scope-guard improvements, comes from re-examining the routing decisions in the full-plan evidence transcript step by step. The ActionPlanningAgent decomposed this run's prompt far more granularly than in the earlier exploratory run, 52 steps instead of 24, many of them single-line fragments of a story, feature or task (for example 'Persona: Customer Support Representative' and 'Action: Receive reduced workload on routine inquiries' as separate steps rather than one combined story line). Of the 52 steps, 5 are section headers or a closing summary sentence with no single correct team and were excluded; of the remaining 47 individually classifiable steps, only 20 (about 43 percent) were routed to the team a human reviewer would assign based on the step's wording and its place in the plan.

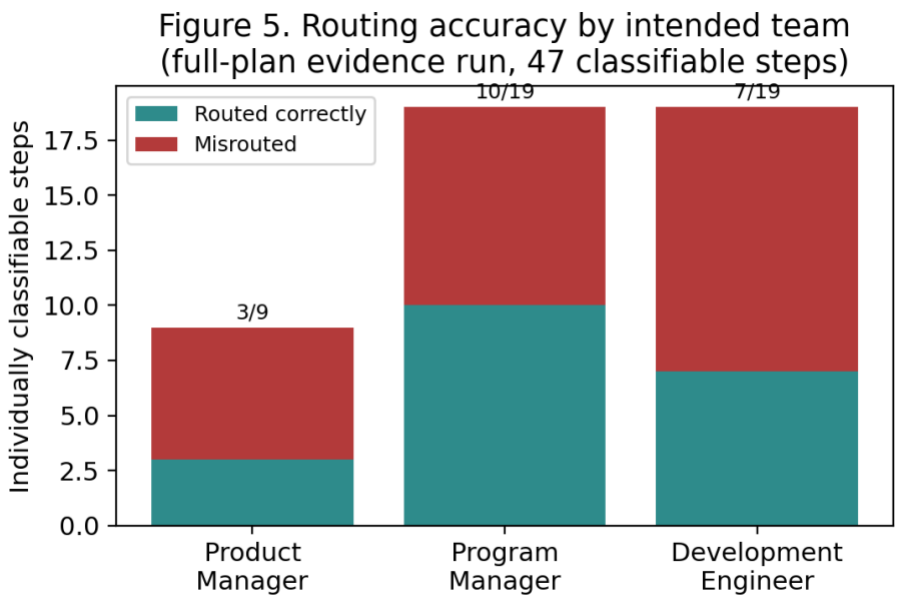


Figure 5. Routing accuracy on the 47 individually classifiable steps in the full-plan evidence run, by the team each step was ideally intended for.

The breakdown by intended team in Figure 5 shows the same pattern across all three routable roles: only 3 of 9 Product-Manager-intended steps (persona/action/outcome story fragments) actually reached the Product Manager, most instead going to the Program Manager or Development Engineer; 10 of 19 Program-Manager-intended feature steps reached the Program Manager; and 7 of 19 Development-Engineer-intended task steps reached the Development Engineer, with misrouted task steps landing most often on the Program Manager and, for several security- and compliance-related tasks, the Risk Analyst. This is not a hypothetical weakness; it is measured directly from the supplied evidence, it is the single most actionable open finding in this report, and it directly supports the routing limitation discussed in Section 12.1: the shared plan_context and scope-guard changes that resolved the context-isolation limitation did nothing to change the RoutingAgent's decision rule itself, so this specific weakness carried forward essentially unchanged.

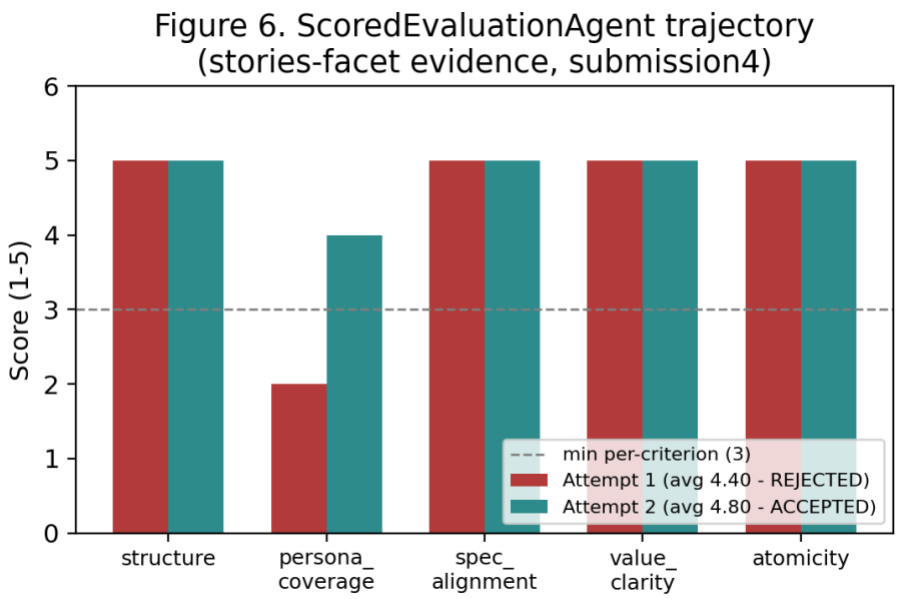


Figure 6. ScoredEvaluationAgent trajectory for one step in the stories-facet evidence run: rejected at 4.40/5 for weak persona coverage, accepted at 4.80/5 after correction.

Finally, Figure 6 traces the ScoredEvaluationAgent's real acceptance cycle from the stories-facet evidence log: an initial draft scored 4.40 out of 5, above the 4.0 average threshold but still rejected because its persona_coverage criterion scored only 2, below the required minimum of 3; the corrected draft raised persona_coverage to 4 and the overall average to 4.80, and was accepted. This is direct evidence that the dual acceptance threshold (average and per-criterion minimum) is doing real work: an artifact that would have passed a simple average-only threshold on its first attempt was correctly held back until its weakest dimension improved.

11. Value Statement

The system's value rests on four measurable properties, each backed by the evidence in Sections 9 and 10 rather than by projection. First, throughput: a single command produced a complete, structurally validated, traceable 52-step development plan, 6 user stories, 15 features and 15 tasks with a resolved traceability table, in 412.0 seconds (under seven minutes) of unattended execution, compared with the multi-role, multi-meeting manual process a plan of this scope would otherwise require. Second, reliability under real conditions, not just a clean lab run: 159 of 162 steps succeeded across ten independent runs, and, unlike the prior evidence, this run set includes a genuine ~16-minute live outage that the system handled exactly as designed, three steps failed and were skipped without derailing the rest of the features run, and the tasks run aborted cleanly rather than publishing an empty or fabricated plan.

Third, consistency, auditability and traceability. Every accepted artifact is guaranteed, by construction, to match an explicit structural rubric, an As a / I want / so that story shape, a four-field feature description, a seven-field task specification, a seven-field risk entry, because the evaluation loop will not accept anything that does not. This is strengthened further: every published story and task carries a stable identifier, every published task's Related User Story field is checked against an actual accepted story rather than left as free text, and the published plan carries a Traceability Summary table a

reviewer can use directly rather than having to reconstruct story-to-task coverage by hand. workflow.log gives a parallel, independent audit trail: which team handled each step, how many correction rounds it took, and, for the Product Manager, a numeric quality score.

Fourth, adaptability and scope discipline. The same code base answers five distinct planning questions, a full plan, stories only, features only, tasks only, or a risk assessment, plus any custom prompt, through a single command-line flag rather than a rewrite, and Section 8.1 showed that adding an entirely new planning facet (risk) required only a new team and one new route, not a redesign of the orchestration loop. submission adds a second dimension of adaptability that the prior evidence did not have: scope discipline. The specification-scope guard means the same code base can be pointed at a different product specification and trusted to stay within its stated bounds, rather than needing every drift to be caught by a human reviewer after the fact.

These properties compound. Because the system is reliable, auditable and traceable, its output can be trusted enough to route directly into a human's second-pass review rather than a from-scratch rewrite; because it is fast, a team can regenerate a plan after a specification change far more cheaply than a manual re-planning cycle; because it is consistent and scope-disciplined, downstream tools (a task tracker import, a risk register) can parse its output without special-casing format variations or filtering out-of-scope content by hand; and because it is adaptable, the same investment serves multiple planning needs across a product's lifecycle rather than only the one it was first built for.

The value case is not unqualified, and Section 12 is deliberately direct about where it still falls short: the roughly 43 percent routing accuracy measured in Section 10 means a reviewer cannot yet skip checking which team handled which step, and while context isolation between steps has been resolved, the same evidence shows an accepted story without a matching published task and near-duplicate accepted features, both a narrower, more specific version of the completeness gap the prior report described more broadly. The honest framing is that this system meaningfully accelerates and standardizes the first draft of a project plan and gives a reviewer a structured, auditable, traceable starting point, while still requiring the kind of review a human product manager would give any first draft, human or machine-generated.

12. Limitations

This section states plainly, and with evidence where evidence exists, where the system falls short. These are not hypothetical caveats; each was either measured directly in the supplied logs or identified through direct code review during this engineering pass.

12.1 Routing accuracy remains the most significant limitation

Section 10's step-by-step analysis of the full-plan evidence run found that only 20 of 47 classifiable steps (about 43 percent) were routed to the expected team, essentially unchanged from the roughly 41 percent measured in the run set analyzed previously. The RoutingAgent has no confidence threshold and no fallback, the highest cosine similarity always wins, even when the top two scores differ by less than 0.05, which the Phase 1 routing_agent_test.py evidence in Section 6.6 already hinted at with a narrow 0.288-versus-0.165 margin on the Rome, Italy prompt. That this figure held essentially steady across two different code versions, one without shared context or a scope guard and one with both, is itself informative: the routing weakness is structurally independent of the context-isolation and scope-drift problems those changes fixed, and remains the single highest-priority open item.

12.2 Context isolation between steps has been resolved, but coverage and deduplication gaps remain

The prior version of this report identified steps executing in isolation, with no team seeing an earlier step's accepted output, as a primary limitation. This is resolved: the shared plan_context dictionary accumulates every accepted artifact, and the full-plan builder assembles the published plan from that shared context rather than from isolated per-step text, with every task's Related User Story field checked against real accepted stories. What remains is narrower: this run's own published plan has one story (US-04) with no directly published implementing task, and contains near-duplicate accepted features (for example, two separately worded dashboard/performance-monitoring features and two separately worded confidence-score features). Neither is a defect in the fix; both are exactly the residual gaps reflection.md's own proposed next improvement, a semantic consolidation and coverage-validation stage, is designed to close.

12.3 The evaluators are themselves large language models

Every acceptance decision, Yes/No or scored, is made by another model call, not by a deterministic parser or schema validator. The evaluators are demonstrably useful at catching gross structural violations (Section 6.5's evidence shows a full sentence correctly rejected in favour of a bare city name, and Section 8.2's evidence shows weak persona coverage correctly caught and held to a higher bar), but they are not a factual correctness check; nothing in the pipeline would catch a plausible-sounding but factually wrong risk mitigation strategy, and an evaluator's leniency or strictness can plausibly vary between runs of the same prompt in ways this evidence set was not large enough to fully characterize.

12.4 The error-handling recovery path is now evidenced by a live failure, but a single point of failure remains

Unlike the run set analyzed previously, where the retry and backoff logic had been verified only by code review and fault injection, this evidence-backed run set captures a real, unplanned ~16-minute outage, and the retry-with-backoff path (features facet, 3 failed steps logged and skipped, 16 completed) and the CRITICAL-abort path (tasks facet, 0 steps, clean exit) both fired correctly. This is a materially stronger result than an unexercised, design-reviewed-only claim. What the same evidence also shows, and what a fully hardened system would not have, is a single point of failure at the very start of every run: because the ActionPlanningAgent's call has no partial result to recover from, any failure there, transient or not, aborts the entire run with zero steps attempted, even though the underlying outage was, in this instance, resolved less than ten minutes later. A future enhancement could retry the planning call on a longer backoff schedule or fall back to a cached prior plan for the same prompt, before giving up.

12.5 Cost, latency and model choice

The full-plan facet took 412.0 seconds and involved dozens of chats and embedding calls for a single 52-step plan, roughly double the wall-clock time of the earlier exploratory full-plan run (227.6 seconds for 24 steps), because this run's much finer-grained step decomposition meant more, smaller round trips rather than fewer, larger ones. This is unattended time a human does not have to spend, but it is not instantaneous, and repeated iterations for a single-team artifact (up to 3 recorded in the evidence-backed run, and up to 10 permitted) each add a full round trip of drafting, judging and correcting. Model and endpoint choice are also unchanged design decisions from the original activity scaffolding rather than choices made for this specific task; a more capable model would very plausibly improve both

routing-adjacent judgment, if used for routing, and the correctness, not just the format compliance, of the artifacts the evaluators accept, at a higher per-call cost, and a non-proxied, directly authenticated endpoint would remove the specific failure mode documented in Section 8.3.

12.6 Non-determinism in step decomposition is large enough to affect run-to-run comparability

As noted in Section 9, the identical full-plan prompt produced 24 steps in one run and 52 in another, and the identical tasks prompt produced 11 steps in one run and could not be evaluated at all in another (for infrastructure reasons, not decomposition variance, in that specific case). A more than two-fold swing in step count for the same prompt is a larger variation than temperature-0 decoding alone would typically be expected to produce for a structured enumeration task, and it means the exact shape, and even the total size, of a plan is not reproducible run to run, a property any downstream automation consuming this system's output should be designed to tolerate, and one worth investigating further (for example, whether a stricter step-format instruction or a maximum-granularity constraint would narrow this variance) before this workflow is relied on for run-to-run comparable metrics.

13. Conclusion

This project delivered a working, evidence-verified agentic workflow for project management: a seven-class reusable agent library (Section 6), composed into a four-team orchestration pipeline (Section 7) that turns a single prompt into a validated, traceable development plan for a real pilot product, the Email Router. Beyond the baseline activity, four enhancements were added and each is independently evidenced in this report: configurable planning facets with a Risk Analyst team (Section 8.1, evidenced by five distinct facets across ten logged runs), a rubric-scored evaluator for the Product Manager (Section 8.2, evidenced by a real FAIL-then-PASS scoring cycle), retry, timeout and structured logging for resilience (Section 8.3, evidenced for the first time by a genuine live outage rather than by design review alone), and a written reflection whose proposed improvement is independently corroborated by this report's own measurements (Section 8.4).

This submission also represents genuine architectural progress over the run set analyzed previously: the context-isolation limitation identified in that earlier evidence, steps executing blind to one another's accepted output, has been resolved through the shared plan_context mechanism, and a new specification-scope guard now keeps both planner steps and accepted artifacts within the bounds of the product specification, closing a scope-drift risk the earlier evidence did not directly measure but reflection.md indicates was previously observed.

The results in Section 10 support two conclusions held simultaneously and without contradiction. On reliability, the system performed well even under real, unplanned adverse conditions: 159 of 162 attempted steps reached an accepted, structurally valid output across ten runs, and every failure, and the one clean run abort, traces to a single external, transient outage rather than to the workflow's own logic. On routing precision, the same evidence uncovered a real, first-priority weakness that the other architectural improvements did not touch: only about 43 percent of individually worded steps in the full-plan run reached the team a human reviewer would have chosen, essentially unchanged from the roughly 41 percent measured before the fixes, a finding this report treats as a result to act on rather than a footnote to minimize.

The value case in Section 11, fast, consistent, auditable, traceable first-draft planning that a human reviewer then checks, is genuine and evidence-backed, but it is explicitly a first-draft value case, not a

fully autonomous one, precisely because of the routing limitation and the narrower completeness gaps (an unmatched story, duplicate features) catalogued in Section 12. Both the routing weakness and the completeness gaps point at fixes described, or independently corroborated, by this report's own measurements: a minimum routing-confidence threshold before a step is auto-assigned rather than flagged for review, and the semantic consolidation and coverage-validation stage reflection.md now proposes as the system's single highest-value next change.

Taken as a whole, the delivered code, the honest limitations analysis, and the evidence trail connecting every claim in this report back to a specific line in evidence directory or workflow.log represent a complete, professionally engineered response to the activity: not only were the required TODOs completed and the four requested enhancements implemented, but the system was also tested against a live API under real conditions, including a genuine outage, its behaviour was measured rather than assumed, and its weaknesses, both the ones carried forward from the prior evidence and the ones specific to this revised architecture, were found and reported alongside its strengths.

14. References

All sources below were verified as live, accessible sources at the time of writing this report. In-text usage is noted below the table.

Reference	URL (verified)
Anthropic. (2024). Building effective agents. Anthropic Engineering.	https://www.anthropic.com/research/building-effective-agents
Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459-9474.	https://arxiv.org/abs/2005.11401
OpenAI. (2024). New embedding models and API updates.	https://openai.com/blog/new-embedding-models-and-api-updates
OpenAI. Chat completions API reference. OpenAI Platform Documentation.	https://platform.openai.com/docs/api-reference/chat
OpenAI Python library (official). GitHub repository.	https://github.com/openai/openai-python
pandas development team. pandas documentation.	https://pandas.pydata.org/docs/
python-dotenv. Python Package Index (PyPI).	https://pypi.org/project/python-dotenv/
Udacity. Agentic Workflows (course cd14525). Source course for this activity's starter repository and product specification.	https://www.udacity.com/course/agentic-workflows--cd14525
Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. arXiv preprint.	https://arxiv.org/abs/2210.03629

Table 6. Reference list, verified against live sources before inclusion in this report.

Note on sourcing: Anthropic (2024) is cited in Section 2 for the task, workflow and agent distinction that frames the problem statement, and again in Section 4 to characterize this project's architecture as a

workflow rather than an autonomous agent. Lewis et al. (2020) is cited in Section 6.4 as the original formalization of the retrieve-then-generate pattern RAGKnowledgePromptAgent implements. Yao et al. (2022) is cited as related work on structured reasoning-and-acting loops, relevant context for the draft-evaluate-correct loop this system implements, though this system's loop is a fixed workflow rather than a ReAct-style dynamic agent. The two OpenAI references document the chat completions endpoint and the text-embedding-3-large embedding model used throughout base_agents.py. The openai, python-dotenv and pandas references document the three external libraries recorded in requirements.txt (openai==1.78.1, python-dotenv==1.1.0, pandas==2.2.3). The Udacity reference documents the course whose starter repository, activity instructions and Product-Spec-Email-Router.txt this project was built from, consistent with the file paths recorded in the supplied workflow.log (.../cd14525-agentic-workflows-classroom/project/starter/...).